19. Mimari Nedir ve Junior Neden Bilmelidir? Monolit vs. Mikroservis

Erken Dönem Mikroservis Çılgınlığına Kapılmadan Doğru Mimarisi Seçmek
Bölüm 4 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 13 dk

---

🎙️ **NotebookLM & Audio Overview Podcast Notları**

Podcast Tartışma Sorusu:
"Bir şirketin 5 kişilik junior/mid ekibi varken neden Netflix gibi 200 mikroservis yapmaya çalışması ölümcül bir hatadır?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Yazılım mimarisi, projenin ilerleyen aşamalarında değiştirilmesi pahalı olan kararlar bütünüdür.
- 📌 Monolit mimari kötü kod demek değildir; modüler bir monolit başlangıç aşamasındaki ekipler için en hızlı ve güvenli yoldur. - 📌 Yapay zeka araçları tek bir monolit kod tabanında (monorepo) çok daha yüksek bağlam (context) başarısı gösterir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir kahve dükkanı açtığınızı düşünün.

İlk gün dükkanda sadece siz ve bir arkadaşınız var. Şöyle bir plan yapar mısınız: - Kahve çekirdeği öğütme departmanı ayrı bir binada - Süt ısıtma departmanı arka sokakta - Kasa ayrı bir binada - Her departman birbirine telsizle (HTTP/gRPC) sipariş numarası yollasın!

Böyle yaparsanız bir fincan kahveyi 45 dakikada servis edemezsiniz ve batarsınız! İşte erken aşamada mikroservise bölünmek tam olarak budur.

Oysa tek bir tezgahın arkasında (Monolit) yan yana durursanız, birbirinizle konuşarak saniyede 1 kahve çıkarırsınız. Dükkanınız Starbucks gibi 1000 şubeye ulaştığında departmanları ayırmayı düşünürsünüz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Monolit vs Mikroservis Karşılaştırma Matrisi:

Kriter	Modüler Monolit (Modular Monolith)	Dağıtık Mikroservisler (Microservices)
Geliştirme Hızı	Çok yüksek (Tek repo, tek derleme)	Düşük (Servisler arası sözleşmeler, RPC)
Dağıtım (Deployment)	Basit (Tek bir Docker imajı / sunucu)	Karmaşık (Kubernetes, servis ağı, API Gateway)
Hata Ayıklama (Debug)	Kolay (Tek stack trace, yerel breakpoint)	Zor (Distributed tracing, Jaeger, correlation ID)
Ölçeklenme	Dikey ölçeklenme (CPU/RAM artırma)	Yatay bağımsız ölçeklenme (servis bazlı)
AI Uyumluluğu	Mükemmel (Tüm modeller ve tipler aynı bağlamda)	Parçalı (Servis sınırları prompt sınırlarını zorlar)

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Baş Sistem Mimarı (Chief Architect)
Yeni kurulan bir e-ticaret platformunun teknik mimarisini tartışıyoruz.
Ekip büyüklüğümüz: 4 Junior, 1 Kıdemli Mühendis.

## GÖREV:
Mikroservis mi yoksa Modüler Monolit mi tercih etmeliyiz?
Lütfen şu 4 başlık altında net bir karar raporu hazırla:
1. Geliştirici üretkenliği ve yerel ortam kurulum süresi
2. Operasyonel maliyet (AWS / DevOps altyapısı)
3. Dağıtık veri tutarlılığı ve distributed transaction riskleri
4. Nihai mimari tavsiyen ve geçiş stratejisi
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Mikroservis Çılgınlığı: Basit bir sipariş için 3 farklı HTTP servisini çağırmak
// Ağ gecikmesi, servis kesintisi ve tutarsızlık riski tavan!
export async function createOrderMicroservices(userId: string, itemId: string) {
  // 1. Kullanıcı servisi
  const user = await fetch(`http://user-service/users/${userId}`).then(r =>
r.json());
  // 2. Stok servisi (Eğer çökerse ne olacak?)
  const stock = await fetch(`http://inventory-service/items/${itemId}`).then(r =>
r.json());
  // 3. Ödeme servisi (Para çekilip sipariş servisi çökerse para buharlaşır!)
  const payment = await fetch(`http://payment-service/charge`, { method:
'POST' }).then(r => r.json());

  return { success: true };
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
// Modüler Monolit: Aynı işlem belleğinde (in-memory) güvenli transaction
export class OrderService {
  constructor(
    private readonly userRepo: UserRepository,
    private readonly inventoryRepo: InventoryRepository,
    private readonly paymentGateway: PaymentGateway,
    private readonly db: DatabaseClient
  ) {}

  async createOrder(userId: string, itemId: string): Promise<OrderResult> {
    // Tüm adımlar tek bir veritabanı transaction'ında atomik yürütülür
    return await this.db.transaction(async (tx) => {
      const user = await this.userRepo.findById(userId, tx);
      if (!user) throw new NotFoundError("Kullanıcı bulunamadı.");

      const item = await this.inventoryRepo.reserveItem(itemId, tx);
      if (!item.inStock) throw new StockError("Ürün stokta kalmadı.");

      const charge = await this.paymentGateway.authorize(user, item.price);
      const order = await tx.orders.create({ data: { userId, itemId, chargeId:
charge.id } });

      return { success: true, orderId: order.id };
    });
  }
}
```

💡 **Kod Analizi:** Modüler monolit sayesinde 3 farklı ağ çağrısı (network round-trip) ve servis çökme riski yerine, tek bir veritabanı işlemiyle atomik başarı garanti altına alınır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Ekipte tek bir DevOps mühendisi bile yokken Kubernetes ve 15 mikroservis kurmaya çalışmak.
  - **Çözüm:** İlk 1-2 yıl modüler monolit ile ilerleyin; iş modeli kanıtlanınca gerektiğinde servisleri ayırın.
- **Tuzak:** Monolit kod yazarken modülleri birbirine çorba yapmak (Spagetti Monolit).
  - **Çözüm:** Monolit içinde katı modül sınırları (User, Order, Inventory) belirleyin; servisler sadece interface üzerinden konuşsun.
- **Tuzak:** Mikroservisler arasında distributed transaction (2PC / Saga) karmaşasını yönetememek.
  - **Çözüm:** Mümkün olan her yerde tek veritabanı ve ACID transaction gücünü kullanın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 19: Mimari Kararlar - Monolit vs Mikroservis Rehberi*

- 🖥️ **Slayt 1:** Yazılım mimarisi nedir ve junior mühendis neden bilmelidir?
- 🖥️ **Slayt 2:** Mikroservis hipotezi: Ne zaman gereklidir, ne zaman felakettir?
- 🖥️ **Slayt 3:** Modüler Monolit anatomisi: Hız ve düzenin birleşimi.
- 🖥️ **Slayt 4:** AI araçlarının monolit projelerdeki bağlam üstünlüğü.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Tek bir monorepo içinde `modules/user` ve `modules/order` yapısını gösterin. Modüllerin birbirinin iç detaylarına doğrudan erişmek yerine dışa açılan servis interface'i üzerinden nasıl konuştuğunu IDE'de gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Mevcut projenizin mimarisini inceleyin. Kodlar katmanlara ve modüllere ayrılmış mı, yoksa tüm fonksiyonlar tek bir devasa klasörde mi toplanmış? 3 maddelik bir analiz çıkarın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Erken aşamadaki (startup) küçük bir yazılım ekibi için 'Modüler Monolit' mimarinin en büyük avantajı nedir? - A) Asla kod yazılmaması - B) Dağıtık ağ karmaşıklığı ve DevOps maliyeti olmadan hızlı geliştirme, kolay hata ayıklama ve atomik transaction imkanı sunması - C) Sadece C++ dilinde derlenebilmesi - D) İnternet gerektirmemesi > **Doğru Yanıt:** B > **Açıklama:** Modüler monolit, mikroservislerin getirdiği ağ gecikmesi ve operasyonel yük olmadan yüksek geliştirici hızı ve veri bütünlüğü sağlar.

Soru 2: Mikroservis mimarisinde karşılaşılan en karmaşık teknik problem hangisidir? - A) CSS yazmak - B) Servisler arası ağ gecikmeleri, kısmi çöküşler (partial failures) ve dağıtık veri tutarlılığı - C) Bilgisayar monitörünün çözünürlüğü - D) Klavye tuşlarının sertliği > **Doğru Yanıt:** B > **Açıklama:** Dağıtık sistemlerde ağ güvenilmezdir. Bir servisin çökmesi veya gecikmesi zincirleme hatalara (cascading failure) yol açabilir.